

# CIF request / logic flow — AXI to MCC

Channel Interface. Module-level sub-blocks by name; only the load-bearing seam signals shown. AXI4 front end -> validate/segment/hash -> ROB -> CDC -> MCC. Handshake shorthand: v/r=valid/ready, f/e=full/empty, oh=one-hot, bc=broadcast, ae=async CDC.

**Spine (read):** AR\_CH -> AR\_REQ\_FIFO -> REQ\_VAL\_R -> BURST\_SPLIT\_R -> R\_ADR\_HASH -> PKT\_FORM\_R -> ROB\_RD -> CH\_R\_REQ\_AFIFO =ae=> MCC

**Spine (write):** AW\_CH/W\_CH -> WR\_PAIR -> REQ\_VAL\_W -> BURST\_SPLIT\_W -> W\_ADR\_HASH -> PKT\_FORM\_W -> ROB\_WR -> CH\_W\_REQ\_AFIFO =ae=> MCC

## 1 Host & AXI ports (single port)

| module    | role                     | interface to next (handshake + meta/data)                                                                              |
|-----------|--------------------------|------------------------------------------------------------------------------------------------------------------------|
| AXI_SLAVE | single AXI4 slave port   | OUT: terminates AR/AW/W (in), R/B (out); 5-channel v/r                                                                 |
| AR_CH     | read-addr channel        | OUT: araddr, arlen, arsize, arid arvalid/arready -> AR_REQ_FIFO                                                        |
| AW_CH     | write-addr channel       | OUT: awaddr, awlen, awsize, awid awvalid/awready -> AW_REQ_FIFO                                                        |
| W_CH      | write-data channel       | OUT: beats wdata, wstrb, wlast wvalid/wready -> WD_SRAM (via WR_SRAM_MGR); descriptor start_ptr, count -> W_BURST_FIFO |
| R_CH      | read-data channel (out)  | IN: rdata, rresp, rlast, rid rvalid/rready <- MERGE_LOGIC                                                              |
| BRESP_CH  | write-resp channel (out) | IN: bresp, bid bvalid/bready <- BRESP_GEN                                                                              |

## 2 Read request path (AR -> MCC)

| module         | role                                                 | interface to next (handshake + meta/data)                            |
|----------------|------------------------------------------------------|----------------------------------------------------------------------|
| AR_REQ_FIFO    | AR buffer (decouple ARREADY)                         | OUT: AR req; f/e -> ARREADY back-pressure -> REQ_VAL_R               |
| REQ_VAL_R      | align/len/legality, compute n_segs                   | OUT: valid req + n_segs -> BURST_SPLIT_R; illegal -> ERR_HDLR_R      |
| ERR_HDLR_R     | read error path                                      | OUT: DECERR/SLVERR -> alloc ROB_RD entry, retire-eligible            |
| BURST_SPLIT_R  | cut burst into row_size segments                     | OUT: per-segment req (1 pkt each) -> R_ADR_HASH                      |
| R_ADR_HASH     | XOR-hash row+BG+bank (col un-hashed)                 | IN: HASH_CFG (bc). OUT: daddr coords -> PKT_FORM_R                   |
| ROB_RD         | read reorder buf start_ptr, exp_pkts, bitmap, status | OUT: rob_index + dbuf base -> PKT_FORM_R; bitmap full -> MERGE_LOGIC |
| RD_SRAM_MGR    | alloc dbuf range at ROB alloc                        | OUT: dbuf_addr -> PKT_FORM_R; free on merge/drain                    |
| PKT_FORM_R     | build read packet                                    | OUT: rob_index, dbuf_addr, daddr, beats -> CH_R_PKT_FIFO             |
| CH_R_PKT_FIFO  | per-channel pkt stage                                | OUT: read pkt; f/e -> CH_R_REQ_AFIFO                                 |
| CH_R_REQ_AFIFO | CDC CIF->MCC                                         | OUT: read req pkt v/r (ae) -> MCC                                    |

## 3 Write request path (AW + W -> MCC)

| module         | role                                                             | interface to next (handshake + meta/data)                                  |
|----------------|------------------------------------------------------------------|----------------------------------------------------------------------------|
| AW_REQ_FIFO    | AW buffer                                                        | OUT: AW req f/e -> WR_PAIR                                                 |
| WR_SRAM_MGR    | alloc per-burst slot-vector on W arrival                         | OUT: wd_slot[] -> WD_SRAM write + W_BURST_FIFO; free per pkt on completion |
| W_BURST_FIFO   | write-data descriptor start_ptr, count (not beats)               | OUT: descriptor -> WR_PAIR                                                 |
| WR_PAIR        | pair AW addr with W descriptor                                   | OUT: paired write req addr, wd_slot, count -> REQ_VAL_W                    |
| REQ_VAL_W      | legality + segmentation (n_segs)                                 | OUT: valid req -> BURST_SPLIT_W; illegal -> ERR_HDLR_W                     |
| ERR_HDLR_W     | write error path                                                 | OUT: DECERR/SLVERR -> alloc ROB_WR entry, retire-eligible                  |
| BURST_SPLIT_W  | split burst, map seg -> WD_SRAM slot                             | OUT: per-segment req -> W_ADR_HASH                                         |
| W_ADR_HASH     | XOR-hash row+BG+bank                                             | IN: HASH_CFG (bc). OUT: daddr -> PKT_FORM_W                                |
| ROB_WR         | write reorder buf; SLOT_FIFO + RETIRE_LOGIC (restore AXID order) | OUT: rob_index -> PKT_FORM_W; retire -> BRESP_GEN                          |
| PKT_FORM_W     | build write packet                                               | OUT: rob_index, wd_slot, daddr, beats -> CH_W_PKT_FIFO                     |
| CH_W_PKT_FIFO  | per-channel pkt stage                                            | OUT: write pkt f/e -> CH_W_REQ_AFIFO                                       |
| CH_W_REQ_AFIFO | CDC CIF->MCC                                                     | OUT: write req pkt v/r (ae) -> MCC                                         |

## 4 Data SRAM & managers (dual-domain: CIF + MCC ports)

| module  | role                   | interface to next (handshake + meta/data)                                                                          |
|---------|------------------------|--------------------------------------------------------------------------------------------------------------------|
| RD_SRAM | 512b/line read buffer  | IN: MCC writes returned read data @ dbuf_addr (MC port). OUT: CIF reads in segment order -> MERGE_LOGIC (CIF port) |
| WD_SRAM | 512b/line write buffer | IN: CIF writes W beats @ wd_slot (CIF port). OUT: MCC reads @ wd_slot to drive DFI (MC port)                       |

## 5 Response path (MCC -> master)

| module          | role                                          | interface to next (handshake + meta/data)                           |
|-----------------|-----------------------------------------------|---------------------------------------------------------------------|
| CH_R_RESP_AFIFO | CDC MCC->CIF read completion                  | OUT: rob_index, pkt_num, status v/r (ae) -> ROB_RD                  |
| ROB_RD (return) | set bitmap bit per pkt                        | OUT: bitmap full -> trigger MERGE_LOGIC                             |
| MERGE_LOGIC     | wait all pkts, drain RD_SRAM in segment order | OUT: rdata, rresp, rlast v/r -> R_CH; free dbuf range (RD_SRAM_MGR) |

| module          | role                          | interface to next (handshake + meta/data)                   |
|-----------------|-------------------------------|-------------------------------------------------------------|
| CH_W_RESP_AFIFO | CDC MCC->CIF write completion | OUT: rob_index, pkt_num, status v/r (ae) -> ROB_WR          |
| ROB_WR (retire) | mark done, restore AXID order | OUT: retired write -> BRESP_GEN; free wd_slot (WR_SRAM_MGR) |
| BRESP_GEN       | form AXI write response       | OUT: bresp, bid bvalid/bready -> BRESP_CH                   |

## 6 Config (APB / CSR — broadcast, off the request path)

| module             | role                                             | interface to next (handshake + meta/data)            |
|--------------------|--------------------------------------------------|------------------------------------------------------|
| APB_SLAVE          | host register access                             | OUT: reg r/w -> CSR                                  |
| CSR                | control/status word file                         | OUT: CFG_REG, ADR_CFG, GEOM_CFG, HASH_CFG (bc)       |
| CFG_REG / GEOM_CFG | enable/mode + ranks/BG/banks/rows/cols, row_size | OUT (bc): -> REQ_VAL, BURST_SPLIT (n_segs, row_size) |
| ADR_CFG / HASH_CFG | field widths/positions + XOR taps                | OUT (bc): -> R/W_ADR_HASH, address decode            |

**Seams to MCC:** req out = CH\_R/W\_REQ\_AFIFO ({rob\_index, daddr, dbuf\_addr|wd\_slot, beats}); resp in = CH\_R/W\_RESP\_AFIFO ({rob\_index, pkt\_num, status}); data shared via RD\_SRAM (MCC writes) + WD\_SRAM (MCC reads). Column left un-hashed so MCC address-slice stays pure. Mirrors `mc_request_flow.pdf` on the MCC side.

# MC-core request / logic flow — AXI to DFI

One channel, DDR5. Module-level sub-blocks by name; only the load-bearing seam signals shown. Per-rank lane = ingress..rank-arb, replicated xN\_RANKS; L3 + back-end + scoreboard + maint shared. Handshake shorthand: **v/r**=valid/ready, **f/e**=full/empty, **oh**=one-hot grant, **bc**=broadcast (no handshake), **ae**=async CDC FIFO.

**Spine:** CIF =ae=> bp\_FIFO -> WLR -> LOOKAHEAD -> BANK\_DEC -> [BANK\_CELL x32 + FSM + ELIG] -> CMD\_ARB -> BGB/BG\_ARB -> RANK\_ARB -> L3 -> CMD\_PHASER -> DFI\_ENC -> PHY

## 1 Command / request path (per rank lane, xN\_RANKS)

| module                    | role                                                                  | interface to next (handshake + meta/data)                                                                                                       |
|---------------------------|-----------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| CIF (host)                | segment <=16-beat pkts, hash addr->daddr, alloc rob_index + sram slot | OUT: reqrob_index, daddr, wd_slot dbuf_addr, op v/r over CDC                                                                                    |
| R_REQ_AFIFO / W_REQ_AFIFO | async in, the only true CDC                                           | OUT: same rec, synced to mc_clk; v/r (ae)                                                                                                       |
| RD_BP_FIFO / WR_BP_FIFO   | per-rank back-pressure, split by dir (depth 16)                       | OUT: rob_index, phy_addr, wr_data_prt; flags f/partial_f -> TURN_CTRL, e -> arb skip                                                            |
| TURN_CTRL + WLR_SEL       | weighted R/W batch, one merge point                                   | OUT: one combined stream + batch_dir; ctrl rd/wr_select (mux)                                                                                   |
| LOOKAHEAD                 | 4-entry bypass window (e0..e3)                                        | IN: bank_rdy[N_BANKS-1:0] (bc). OUT: rob_index, phy_addr, op head; shift on sel                                                                 |
| BANK_DEC                  | 1-of-N_BANKS decode rank,bank                                         | OUT: per-bank push strobe; consumes bank_rdy                                                                                                    |
| BANK_REQ_REG[r,b]         | 1-deep payload hold                                                   | OUT (at CAS-commit, via grant_oh): op, rob_index, col, sram_addr -> CMD_PHASER                                                                  |
| BANK_STATUS_REG[r,b]      | state[4], open_row, valid, pre_pending                                | OUT: fields -> ROW_COMPARATOR + ELIG (bc read)                                                                                                  |
| ROW_COMPARATOR[r,b]       | pkt.row vs open_row                                                   | OUT: is_hit, need_pre/act/cas -> ELIG                                                                                                           |
| BANK_ELIG[r,b]            | need_* & can_* fold                                                   | IN: can_* (scoreboard bc), stall_acts. OUT: bank_go, bank_cmd(cas/act/pre), is_hit -> CMD_ARB                                                   |
| BANK_FSM[r,b]             | 6-state cmd-progress FSM                                              | IN: grant_* (issue-edge), can_cas/can_act, gate_rfc/rfm. OUT: state, cas_complete, bank_busy, row_valid, act/cas/pre_set (scoreboard writeback) |
| CMD_ARB[r,b]              | tier-1: pick cmd for bank                                             | IN: CAS_W/ACT_W/PRE_W. OUT: go, cmd, is_hit -> BGB_ARB                                                                                          |
| BGB_ARB / BG_ARB          | bank-in-BG (4->1), BG-in-rank (8->1)                                  | OUT: per-rank per-class candidate; BG_ARB weight = diff-from-last_cas_bg                                                                        |
| RANK_ARB (act/cas/pre)    | rank_act_abr, rank_cas_abr, pick-2-pre                                | OUT: act_win, cas_win, pre_win[0..1] = rdy, bg, bank -> SCHEDULER                                                                               |
| SCHEDULER + L3            | class selectors (cas_L3 +dqFree/tRTRS, act_L3, pre_L3), pick <=2      | IN: winners + maint flags + turnaround. OUT: 2 slots vld, len, CMD_PACKET; grant_*_oh (pop-on-CAS), fsm_commit, cmd_issued                      |
| CMD_PHASER                | 2-slot cmd_wire (not a fifo)                                          | IN: 2 slots + gear. OUT: cmd_0 -> ENCO, cmd1 -> ENC1; rdy0/rdy1 back-pressure -> scheduler                                                      |
| DFI_CMD_ENCODER x2        | OPCODE_ROM + FIELD_MUX + CS_GEN                                       | IN: record + gear_cfg (always_on / b1_enable). OUT: ENCO->dfi_p0/p1, ENC1->dfi_p2/p3; 1:1 temporal mux (2bit_wrap_cnt)                          |
| DFI CMD PORTS             | per-phase drive                                                       | OUT: dfi_address/cs_n/cke/odt/parity_pN -> PHY -> DRAM                                                                                          |

## 2 Write data path (shared)

| module        | role                          | interface to next (handshake + meta/data)                                                                         |
|---------------|-------------------------------|-------------------------------------------------------------------------------------------------------------------|
| AXI W channel | beats in                      | OUT: wdata, wstrb, wlast v/r                                                                                      |
| WR_ACCUM      | beats -> 512b line, WSTRB->DM | IN: narrow flag (from WR_ROB). OUT: 512b line + DM -> WD_SRAM (one wide write on wlast); alloc line on first beat |
| WD_SRAM       | 512b line store (1 pkt/line)  | IN: write @ wd_slot. OUT: 512b read @ wd_slot (on WL fire)                                                        |
| WL_LAUNCH     | write-latency delay line      | IN (at CAS-commit): wd_slot, rob_index, tphy_wrlat. OUT: dfi_wrdata_en @ cmd+tphy_wrlat; fires WR_SPLIT           |
| WR_SPLIT      | 512b -> 2*gear beats/mc_clk   | IN: 512b, gear. OUT: dfi_wrdata_pN @ ent+tphy_wrdata; wr_done (self-timed, last beat)                             |

## 3 Read data path (shared)

| module        | role                      | interface to next (handshake + meta/data)                                                  |
|---------------|---------------------------|--------------------------------------------------------------------------------------------|
| DFI rddata in | PHY drives                | IN: dfi_rddata_pN + dfi_rddata_valid                                                       |
| RD_CAP        | RL delay line             | IN (at RD-issue): dbuf_addr, rob_index, tphy_rdlat. OUT: capture window enable -> RD_ACCUM |
| RD_ACCUM      | 2*gear beats -> 512b line | OUT: 512b line -> RD_SRAM @ dbuf_addr (one wide write)                                     |
| RD_SRAM       | 512b line store           | OUT: 512b read @ dbuf_addr -> RD_SPLIT                                                     |
| RD_SPLIT      | 512b -> AXI R beats       | OUT: rdata, rlast v/r -> AXI R channel                                                     |

## 4 Completion / tracking (shared)

| module                      | role                                    | interface to next (handshake + meta/data)                                                                                         |
|-----------------------------|-----------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| COMPLETION                  | merge rd_done / wr_done                 | IN: rd_donerob_index,pkt (last beat in RD_SRAM), wr_donerob_index,pkt (self-timed). OUT: rob_index, pkt_num, status -> resp AFIFO |
| R_RESP_AFIFO / W_RESP_AFIFO | async out to CIF                        | OUT: tag-only completion v/r (ae) -> CIF                                                                                          |
| WR_ROB                      | write tracker vld, N, gen_idx, done_cnt | gen_idx -> addr offset (w_phy_addr + gen_idx*64B); retire when done_cnt==N -> COMPLETION                                          |

## 5 Scoreboard (feeds ELIG + arb; not in the data stream)

| module          | role                                                | interface to next (handshake + meta/data)                             |
|-----------------|-----------------------------------------------------|-----------------------------------------------------------------------|
| GC              | 24b free-run, +=gear each<br>mc_clk                 | OUT: GC -> all UPD adders + comparators (bc)                          |
| TIMING_CONST    | JEDEC addends ( 20 x 10b,<br>one copy)              | OUT: t -> every *_UPD adder                                           |
| _UPD            | next_* = GC + const, per<br>scope (bank/BG/rank/ch) | IN: cmd_issued (WE + router). OUT: 13b deadline -> *_TREG             |
| _TREG + CMP     | deadline store + GE compare                         | OUT: can_act/can_cas/can_pre (1b) -> ELIG (bc); can_*_bgs/rank -> arb |
| refi_cnt + debt | 14b down-counter + 3b debt                          | OUT: ref_rdy= debt, ref_force=&debt -> MAINT/scheduler                |
| RAA_cnt         | per-bank ACT counter                                | OUT: rfm_rdy (>=RAAIMT), rfm_force (>=RAAMMT) -> MAINT                |

## 6 Maintenance (peer source; injects, never CAS)

| module    | role                                                 | interface to next (handshake + meta/data)                                                                                                                                                            |
|-----------|------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| MAINT_ENG | INIT / REFRESH / RFM / ZQ<br>/ MRW / MRR sub-engines | IN: cas_complete[banks] (drain done), debt/RAA (scoreboard), MR4 poll. OUT: stall_acts -> banks (quiesce), gate_rfc/rfm -> scoreboard mask, maint cmd -> CMD_PHASER (override), init_done -> DFI_MUX |

**Gaps (revisit):** LOOKAHEAD-to-BANK\_DEC decision in full logic space (bank\_cmp / prio-enc detail); per-rank vs shared boundary of a couple scoreboard slots. Flagged, not blocking. **Config (APB/MR/PHY/FREQ\_FSM)** writes CONFIG\_REGS / MR\_SHADOW / tphy\_\* out-of-band (boot / DVFS-quiesced / MR-poll) — not on the request path; see config\_regs.pdf.